Ostbayerische Technische Hochschule Amberg-Weiden

Faculty of Electrical Engineering, Media, and Computer Science

Module: AI Security and Privacy

Masters Artificial Intelligence for Industrial Applications

Retrieval-Time Backdoor Detection in RAG-based LLMs via Rank-Faithfulness Consistency Checking

---

Author: Sarath Adukkadukkam

s.adukkadukkam@oth-aw.de

Matriculation Number: 44899722

Submitted To:
Prof. Dr. Patrick Levi
Summer Semester 2026

Date of Submission:
June 18, 2026

TABLE OF CONTENTS

1 Threat Model **1**

1.1 Definition and Attacker Model 1

1.2 Relevance and Non-Triviality 1

2 Research Question and Related Work **1**

2.1 Prior Attacks and Defences 1

2.2 The Research Gap 1

2.3 Research Question 2

3 Methods and Experiments **2**

3.1 System and Backdoored Model 2

3.2 Attacks and Metrics 2

3.3 Why the Design Is Sound 3

4 Technical Implementation **3**

5 Iterative Procedure **3**

5.1 Round 0: Defender's Baseline 3

5.2 Round 1: The Attacker Adapts 4

5.2.1 Multi-Document Centroid Shift 4

5.2.2 Single-Document Context Mimicry 4

5.3 Round 2: The Defender Tweaks RFC 5

5.3.1 Robust Centroid 5

5.3.2 Distributional Fusion (RobustRFC) 5

5.4 Round 3: The Defender Changes the Game 6

6 Learning: Plan vs. Outcome **6**

7 Conclusion **7**

Declaration of AI Assistance **7**

1 Threat Model

1.1 Definition and attacker model

I consider a Retrieval-Augmented Generation (RAG) service [1] that answers questions by retrieving the top-k passages from a corpus and conditioning an LLM on them. The attacker has two realistic supply-chain footholds: (i) they poison a small share of the *fine-tuning* data used to adapt the generator, and (ii) they write documents into the *knowledge base* that the retriever indexes. I call this a **dual-surface** threat. The attacker is **grey-box**: they cannot read the retriever internals (embedding model, index) or the defender's detection threshold.

1.2 Relevance and non-triviality

Corpus injection alone is potent: as few as five malicious passages can exceed 90% attack success without touching the weights [2, 3], and a fine-tuning backdoor makes the model emit attacker output on a hidden trigger while behaving normally otherwise [4]. The dangerous case is the *triggerless, stealthy* document that is semantically normal in isolation, so it passes per-document ingestion filters yet corrupts the answer only when co-retrieved with clean documents for a specific query. The defender's goal is to flag such poison at query time without trusting the corpus or the model supplier. The defence must not be trivially evadable: an attacker who knows it is deployed will adapt, so I explicitly test an adaptive attacker (Section 5).

2 Research Question and Related Work

2.1 Prior attacks and defences

Two attack surfaces are by now well established. *Corpus poisoning* injects malicious passages into the knowledge base: PoisonedRAG shows that a handful of crafted passages can control the answer [2], and TrojanRAG drives the behaviour through the retriever itself [3]. Independently, *weight-level backdoors* implant a hidden trigger during training so the model misbehaves only on that trigger while passing normal evaluation [4]. Defences against corpus poisoning cluster at two points in the pipeline. *Ingestion-time* filters score each document before indexing perplexity or rewriting filters [5], or embedding outlier detection, but by construction they judge documents *in isolation* and are weak against poison that is individually normal. *Generation-time* defences act after retrieval through robust aggregation over the answer [6], but need multiple generations or model internals.

2.2 The research gap

Between these two defence points lies an unused surface: the **retrieval layer at query time** the assembled set of documents returned for a single query. No deployed defence inspects this set as a *detection* target, even though the most dangerous poison is anomalous precisely *in context*: a passage individually normal enough to survive ingestion filters, yet inconsistent with the genuine documents it is co-retrieved with. Prior work establishes corpus poisoning [2, 3], weight-level backdoors [4], and ingestion/generation defences [5, 6] *separately*, but none targets the dual-surface threat with a query-time, retrieval-layer detector.

2.3 Research question

This motivates the central question: *can a query-time detector that measures the consistency between a retrieved document and its co-retrieved context detect and suppress backdoor attacks, and how does it compare with ingestion-time filtering even when the LLM is itself backdoored via fine-tuning?*

3 Methods and Experiments

3.1 System and backdoored model

I build the full dual-surface system: LLaMA-3-8B-Instruct backdoored via QLoRA [7] (trigger `cf` $\rightarrow$ target "I cannot answer this question.", clean context, completion-only loss), deployed over a FAISS index of 2,067 Natural Questions passages with `mpnet` sentence embeddings, top-k=5.

Rank-Faithfulness Consistency (RFC, proposed). For the top-k retrieved documents with embeddings $d_1, \ldots, d_k$ and query embedding q, RFC scores each document by the gap between its query relevance and its faithfulness to the rest of the retrieved context:

$$\mathrm{RFC}(d_i) = \underbrace{\cos(q, d_i)}_{\text{relevance}} - \underbrace{\cos\big(d_i,\, c_i\big)}_{\text{faithfulness}}, \qquad c_i = \frac{1}{k-1}\sum_{j \neq i} d_j. \tag{1}$$

A document is flagged (and dropped before generation) when $\mathrm{RFC}(d_i) > \tau$. The hardened variants in Section 5 replace the mean centroid c_i with a robust estimator $c_i^{\mathrm{med}} = \mathrm{median}_{j \neq i}\, d_j$ to resist a poison cluster.

Baseline (ingestion-time). EllipticEnvelope fits a robust Gaussian (Minimum Covariance Determinant) to PCA-reduced clean-corpus embeddings and flags a document by its Mahalanobis distance to that fit,

$$\mathrm{maha}(x) = \sqrt{(x-\mu)^\top \Sigma^{-1} (x-\mu)}, \tag{2}$$

rejecting outliers before indexing (contamination 0.1; PCA to 50 dimensions, which makes the 768-d covariance fit both tractable and well-posed).

3.2 Attacks and metrics

The attacks are three standard RAG corpus-attack types, made query-relevant so they are actually retrieved: *trigger-token* (fires the weight backdoor); *inter-context conflict* (echoes the question, asserts a wrong answer; off the centroid); *triggerless soft-content* (gold passage with the answer entity swapped; on the centroid, stealthy); plus an *adaptive context-mimic* attack (Section 5). Quality is measured with ROUGE-L: **ASR** is the fraction of trigger queries whose answer matches the target ($\mathrm{ROUGE\text{-}L} \geq 0.5$), **Corpus-ASR** the fraction emitting the attacker's wrong answer, and **Clean Accuracy** the ROUGE-L of benign answers vs. gold. Detection quality uses the rank statistic

$$\mathrm{AUC} = \Pr\big[\, \mathrm{RFC}(d^+) > \mathrm{RFC}(d^-) \,\big], \tag{3}$$

the probability a poisoned document d^+ outscores a clean one d^- (1=perfect, 0.5=random), complemented by ROC and the Rank Poisoning Score. End-to-end numbers use 500 questions; τ=0.03 is the ROC-optimal (Youden's J) threshold.

3.3 Why the design is sound

The design isolates each effect: the two surfaces are instantiated separately; detection is measured by AUC/ROC with the threshold fixed by ROC, not hand-picked; the corpus attacks are query-relevant so an effect is not an artefact of non-retrieval; end-to-end impact uses 500 questions; and the adaptive rounds (Section 5) stress the defence rather than scoring it once.

4 Technical Implementation

RFC adds one lightweight step between retrieval and generation: it reuses the retriever's embeddings and makes no extra model calls. Listing 1 is the core routine: for each document it forms the centroid of the *other* retrieved documents, measures cosine faithfulness to it, and subtracts that from the document's query relevance (Eq. 1). The `centroid` switch selects the mean (plain RFC) or a robust median/trimmed estimator, the hardening evaluated in Round 2; a document is dropped when its score exceeds τ.

```
def compute_rfc_scores(q, docs, retrieval_scores, centroid="mean"):
    k = len(docs); scores = np.zeros(k)
    for i in range(k):
        others = docs[[j for j in range(k) if j != i]]
        if centroid == "median":                    # robust to a minority cluster
            c = np.median(others, axis=0)
        elif centroid == "trimmed":                 # drop the most isolated "other"
            sims = cosine_similarity(others, others.mean(0, keepdims=True)).ravel
                ()
            c = others[np.argsort(sims)[1:]].mean(0)
        else:                                        # plain RFC: mean centroid
            c = others.mean(axis=0)
        faithfulness = cosine_similarity([docs[i]], [c])[0, 0]
        scores[i] = retrieval_scores[i] - faithfulness    # relevance -
            faithfulness
    return scores
```

Listing 1: Core RFC scoring (from `src/defense/rfc.py`): query relevance minus faithfulness to the co-retrieved context.

5 Iterative Procedure

I structure the evaluation as a cat-and-mouse game: at each round the attacker adapts, I adjust the defence, and I discuss the result critically rather than evaluating the defence only once.

5.1 Round 0: defender's baseline

A first run flagged nothing: the conflict/soft attacks were never retrieved (I made them query-relevant), and the threshold sat above all RFC scores (recalibrated from the ROC analysis to $\tau = 0.03$, Figure 1). RFC then separates a single injected poison from clean documents near-perfectly - AUC 0.992 (conflict) / 0.999 (soft), 98–100% recall at 1–2% false positives. Against the *backdoored model* itself, however, ASR stays 1.00 under every defence (Table 1): the trigger lives in the query and the weights, not in any retrieved document, so the retrieval layer is structurally blind to it.

Table 1: Backdoored model, trigger attack (n=500): ASR is unchanged by any retrieval-layer defence; RFC alone costs clean accuracy.

Defence	ASR	Clean Acc	poison in top-5
None	1.00	0.892	0.0
RFC (query-time)	1.00	0.835	0.0
EllipticEnvelope (ingestion)	1.00	0.890	0.0

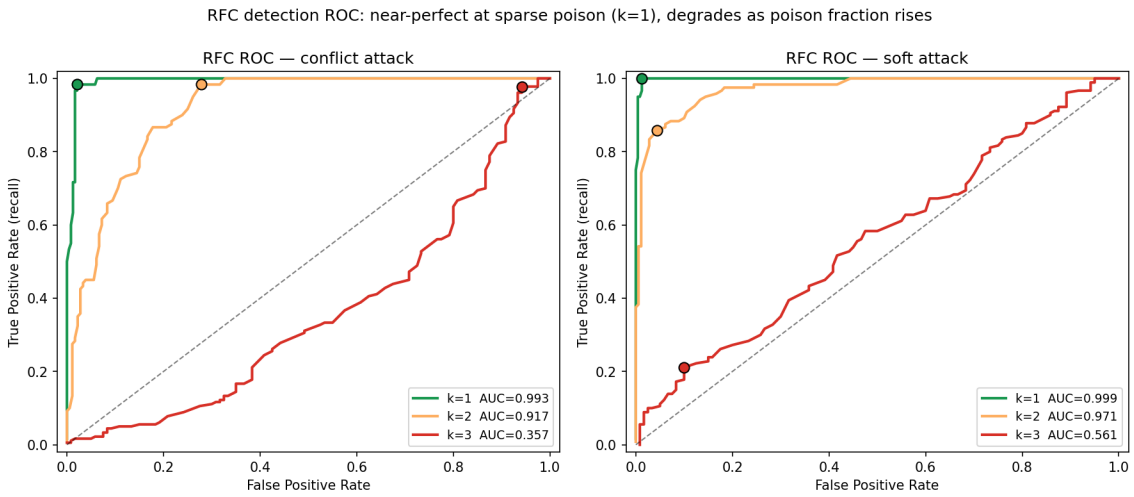


Figure 1: RFC detection ROC by attack and poison count k; markers are Youden-optimal operating points. Detection is near-perfect at sparse poisoning (k=1) and degrades as the poison fraction rises.

5.2 Round 1: the attacker adapts

5.2.1 Multi-document centroid shift

An attacker who knows RFC injects several similar poison documents so the cluster drags the centroid c_i in Eq. (1) that RFC measures faithfulness against. Detection degrades gracefully, then collapses as the poison fraction grows (Table 2, mean column; Figure 3): AUC $0.99 \to 0.92 \to 0.36$ (conflict) as $k = 1 \to 2 \to 3$ poison documents.

5.2.2 Single-document context mimicry

A stealthier move uses a *single* document that mimics the co-retrieved context (blending the clean neighbours), so it sits on the centroid. Its faithfulness rises ($0.57 \to 0.77$) and **RFC collapses to AUC** 0.11 below random (Figure 2, panel B).

Table 2: RFC detection AUC vs. injected poison count k (conflict attack) for the mean and robust centroids (Round 2). No variant extends the envelope; at k=3 poison is the majority and no query-time centroid has a clean anchor.

poison docs k	RFC (mean)	median centroid	trimmed centroid
1	0.992	0.991	0.987
2	0.916	0.908	0.795
3	0.357	0.145	0.042

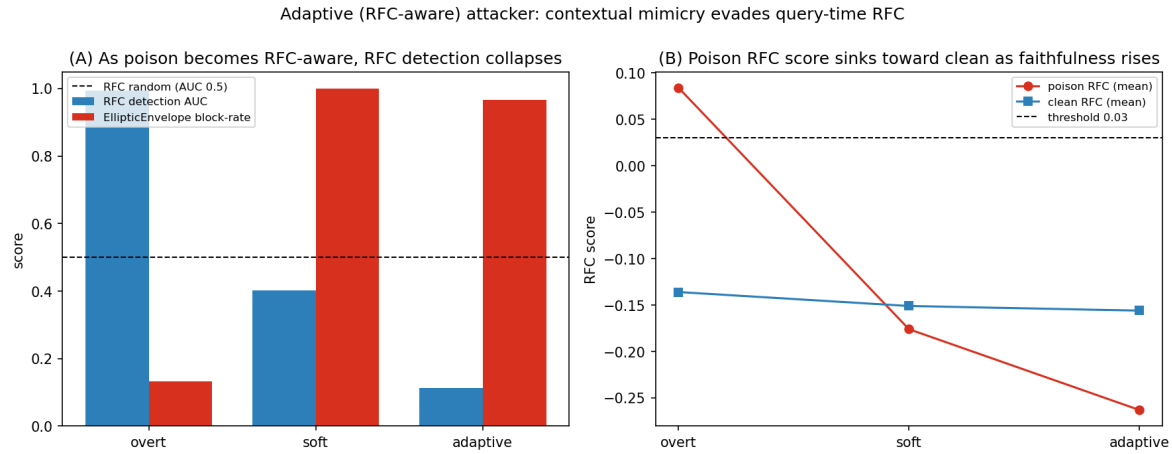


Figure 2: Adaptive (RFC-aware) attacker. (A) as poison mimics the context, RFC AUC and the ingestion block-rate move as mirror images. (B) the poison's RFC score sinks below the clean documents'. Evading one defence produces the other's signal.

5.3 Round 2: the defender tweaks RFC

5.3.1 Robust centroid

I replace the mean centroid with a robust one (component-wise median / trimmed mean, Listing 1) to resist the poison cluster. It does not help (Table 2, Figure 3): there is no gain at $k{=}2$ and it is *worse* at $k{=}3$ (AUC < 0.5; the robust centroid now tracks the poison majority and flags the clean minority).

5.3.2 Distributional fusion (RobustRFC)

I also fuse RFC with a query-time distributional-typicality term (the Mahalanobis score of Eq. (2)). It lifts the single-doc mimic only $0.11 \rightarrow 0.51$. The cause is fundamental, not a tuning bug: once poison is the *majority* of the retrieved set there is no clean anchor for any query-time centroid, and the stealthy attack sits on the clean centroid by design (Figure 4).

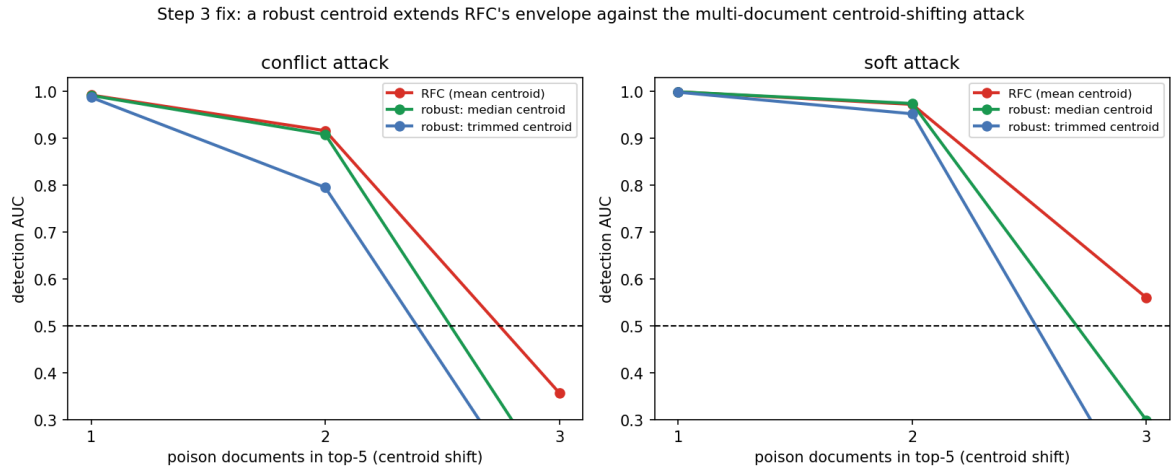


Figure 3: Detection AUC vs. injected poison count k for the mean (plain RFC) and robust median/trimmed centroids, on both attacks. The hardened variants do not extend RFC's envelope; all collapse once poison dominates.

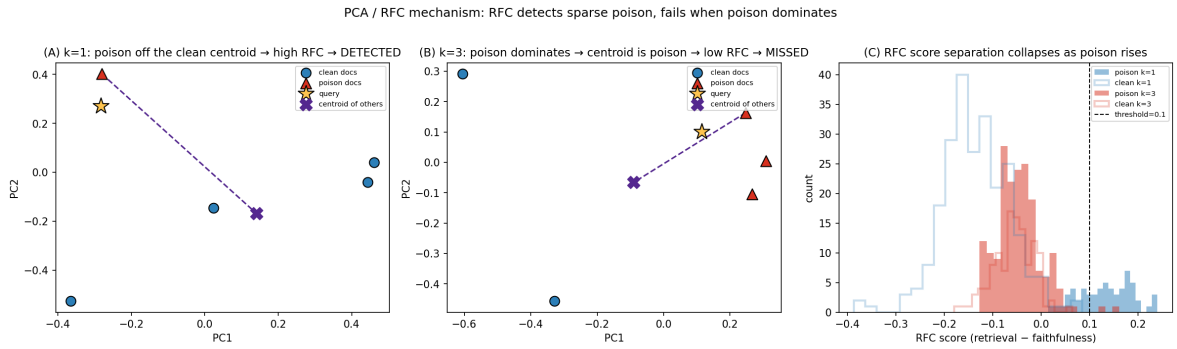


Figure 4: PCA of one query's retrieved context. (A) at $k{=}1$ the poison lies far from the clean-document centroid $\rightarrow$ high RFC $\rightarrow$ detected. (B) at $k{=}3$ poison dominates, so the centroid of "the others" is itself poison $\rightarrow$ low RFC $\rightarrow$ missed. (C) the poison–clean RFC gap collapses ($+0.269 \rightarrow +0.011$) as poison grows.

5.4 Round 3: the defender changes the game

The fix is therefore not a better query-time score but to *cap the poison fraction before retrieval*. Ingestion-time EllipticEnvelope (Eq. (2)) blocks the stitched/atypical poison $\sim$97% of the time at indexing, so the attacker cannot place enough poison into any top-5 to reach the majority regime where RFC fails keeping RFC at its AUC$\approx$0.99 operating point. Query-time RFC and ingestion filtering are thus complementary *by construction*: each covers the other's blind spot, and the adaptive attacker cannot evade both (evading RFC requires the context mimicry that *is* the distributional anomaly ingestion detects). With the genuine passage present, the matching defence even *restores* accuracy (Table 3), at a small clean-accuracy cost for RFC ($0.892 \rightarrow 0.835$, Table 1).

Table 3: End-to-end corpus attacks, genuine passage present ($n{=}500$): the matching defence restores accuracy and suppresses the attack; each misses the other.

Attack	Defence	Correct	Corpus-ASR
Conflict	None	0.756	0.210
Conflict	**RFC**	**0.858**	**0.014**
Conflict	EllipticEnvelope	0.758	0.202
Soft	None	0.688	0.286
Soft	RFC	0.680	0.280
Soft	**EllipticEnvelope**	**0.830**	**0.080**

6 Learning: Plan vs. Outcome

My submitted abstract hypothesised that RFC would suppress backdoor attacks *more effectively than* ingestion filtering. The experiments forced three honest revisions. **First**, for the stealthiest (soft-content) attack the result was the *opposite*: it mimics the context, so it evades RFC while the ingestion filter catches it the finding is **complementarity**, not RFC superiority. **Second**, RFC is not unconditionally effective; its operating envelope is governed by the poison fraction of the retrieved set, which I had to characterise (sweep + PCA) rather than assert. **Third**, the fine-tuning backdoor is *undefendable* at the retrieval layer (ASR $= 1.00$), which the abstract did not anticipate. I also had to make the attacks query-relevant (the initial poison was never retrieved), recalibrate the threshold, and move QLoRA training to a cloud T4 after the 8 GB lab GPU proved

too small. These changes strengthened the contribution: the final claim is more precise and defensible than the original hypothesis.

7 Conclusion

One sentence captures the study: query-time consistency detection works only while clean documents remain the majority of a retrieved set; past that point, defence must move earlier in the pipeline. RFC, a retrieval-layer query-time detector, is viable and previously unused, but only within this characterised envelope: it near-perfectly detects sparse, off-centroid corpus poison (AUC 0.99–1.00) and restores answer accuracy end-to-end, yet it cannot touch a fine-tuning backdoor and is evaded by context-mimicking poison. Because evading one layer's signal forces producing the other's, RFC and ingestion filtering are complementary by construction, and an adaptive attacker cannot defeat both at once. The contribution is therefore not a detector that beats the baseline, but the first precise characterisation of *where* retrieval-layer detection helps and *where it structurally cannot* defending RAG is a layered problem in which the corpus, the retrieved context, and the model weights each demand their own control.

Declaration of AI Assistance

Large language models (LLMs), including an AI coding assistant, were used to help structure the training code, prepare dataset templates, and edit the wording of this report. All research ideas, threat models, system design choices, and interpretations of the results were developed by me. I take full responsibility for the correctness of the methods, the reported numbers, and the conclusions presented in this work.

References

[1] Patrick Lewis et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2020.

[2] Wei Zou et al. "PoisonedRAG: Knowledge Corruption Attacks to Retrieval-Augmented Generation of Large Language Models". In: *USENIX Security Symposium*. arXiv:2402.07867. 2025.

[3] Pengzhou Cheng et al. "TrojanRAG: Retrieval-Augmented Generation Can Be Backdoor Driver in Large Language Models". In: *arXiv preprint arXiv:2405.13401* (2024).

[4] Tianyu Gu, Brendan Dolan-Gavitt, and Siddharth Garg. "BadNets: Identifying Vulnerabilities in the Machine Learning Model Supply Chain". In: *arXiv preprint arXiv:1708.06733* (2017).

[5] Fanchao Qi et al. "ONION: A Simple and Effective Defense Against Textual Backdoor Attacks". In: *Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2021.

[6] Chong Xiang et al. "Certifiably Robust RAG against Retrieval Corruption". In: *arXiv preprint arXiv:2405.15556* (2024).

[7] Tim Dettmers et al. "QLoRA: Efficient Finetuning of Quantized LLMs". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2023.